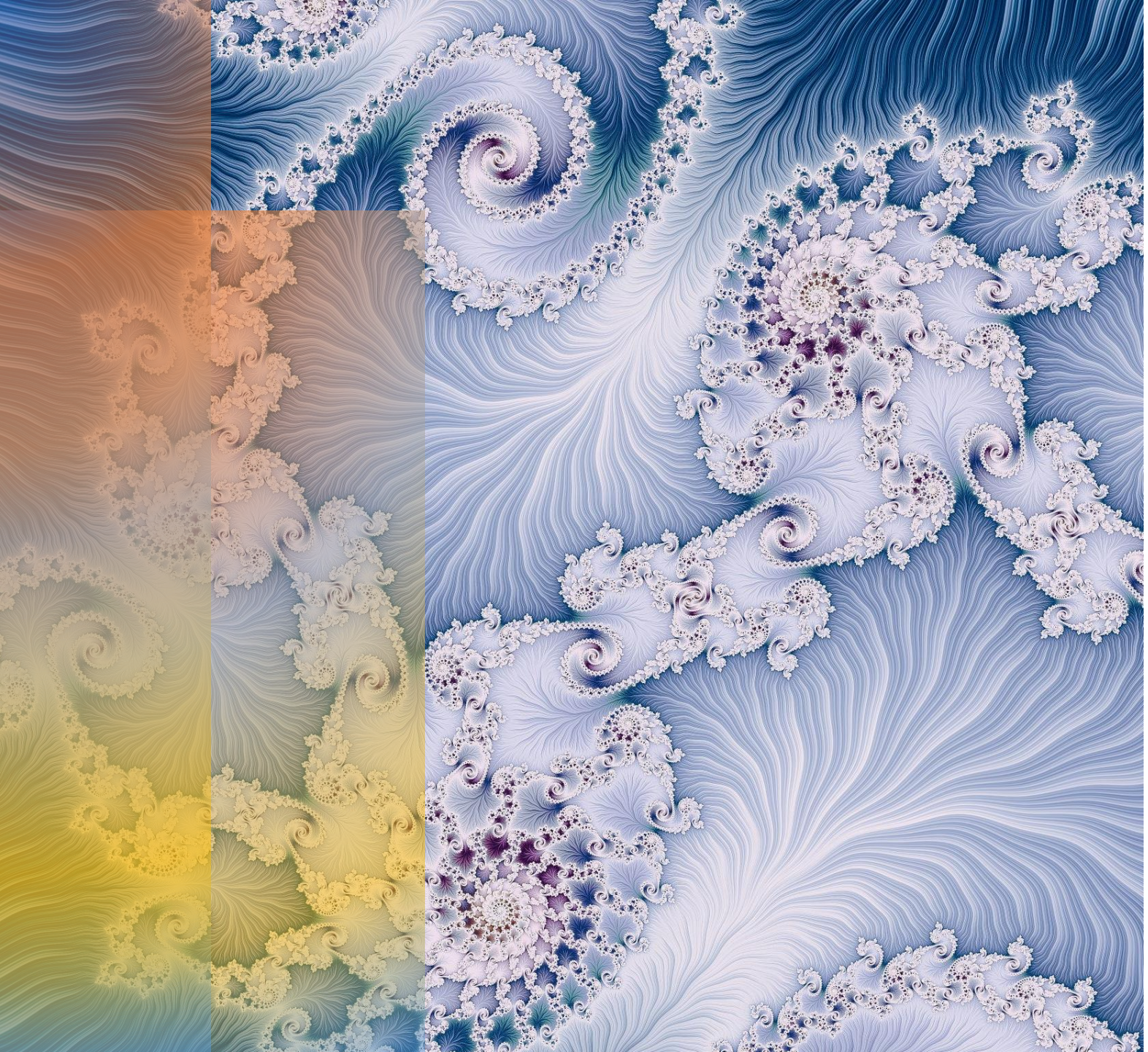


Computer Graphics

Week 4: WebGL (1)

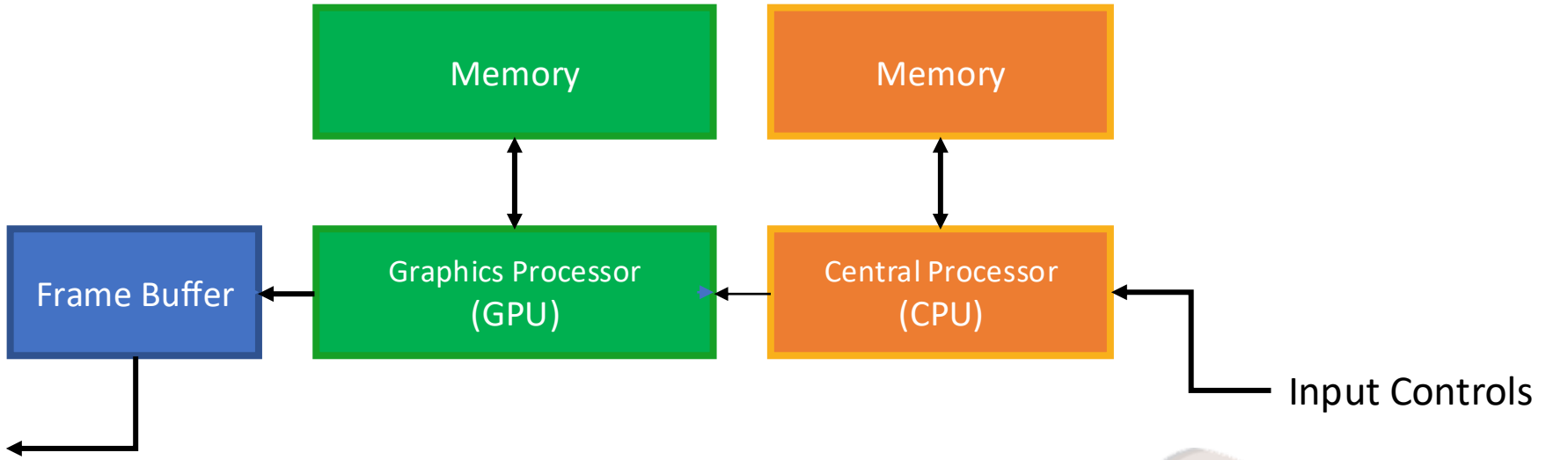
Sub Topics

- Basic Graphics System
- Graphics API
- Graphics Pipeline
- WebGL Introduction
  - Programmable Pipeline
  - GLSL introduction

Basic Graphics System



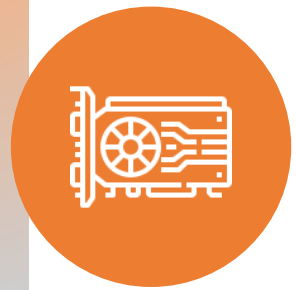
Output Displays



Graphics are rendered
in the frame buffer



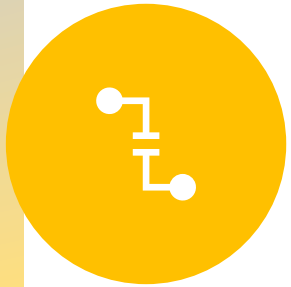
Modern Graphics Library



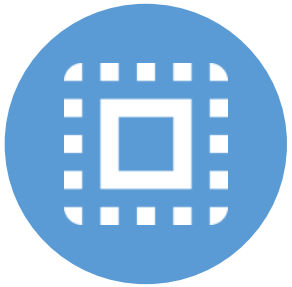
Performance is achieved by GPU (not CPU).



GPU is controlled through programs called shaders



The job of the application (like JavaScript) is to send data to the GPU.



GPU is the one who does the whole rendering process.

Some of Graphics Libraries

OpenGL ES

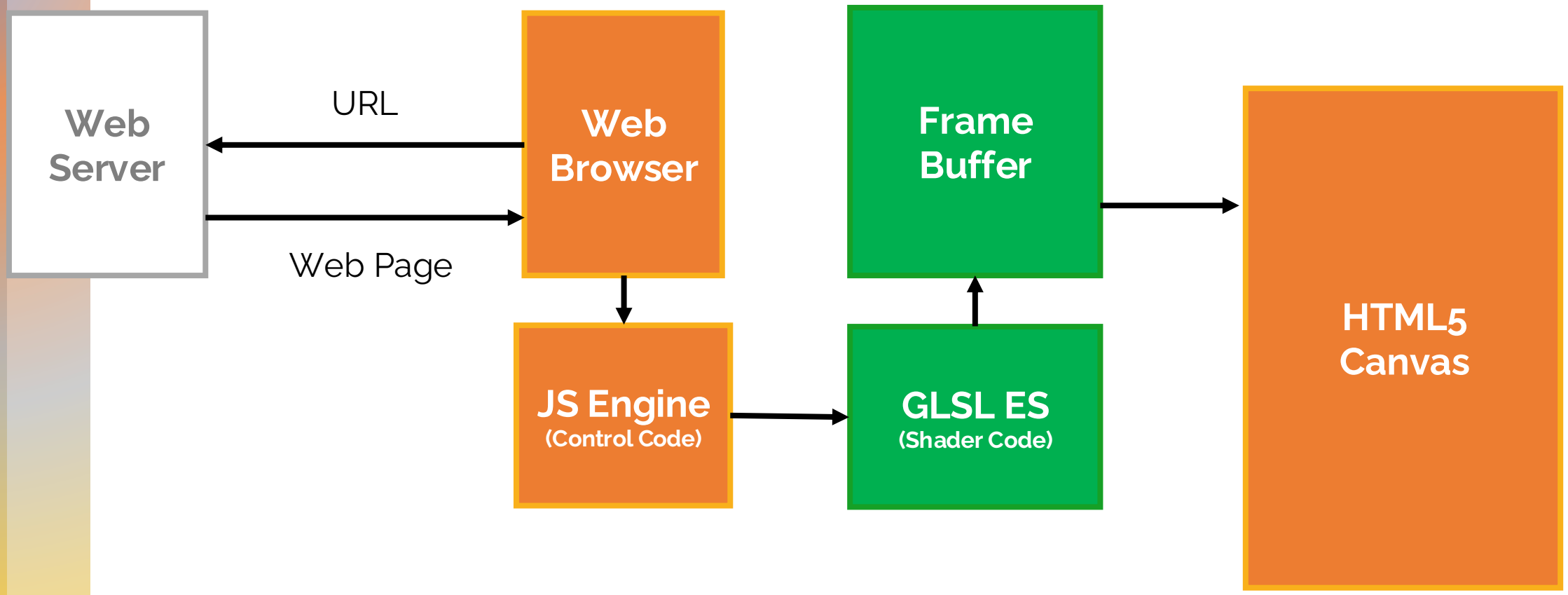
ES = Embedded Systems.
OpenGL ES version 1.0 is a simplification of OpenGL 2.1
OpenGL ES version 2.0 is a simplification of OpenGL 3.1 (shader-based)

WebGL 1.0 / 2.0

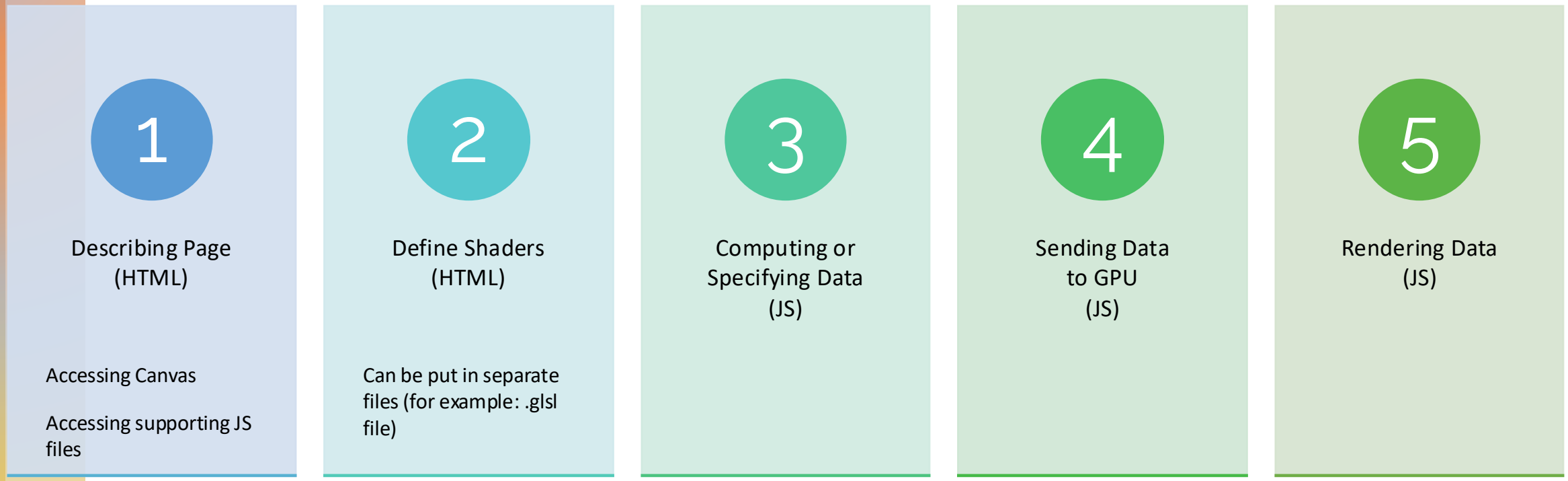
JavaScript Implementation of OpenGL ES version 2.0 / 3.0

WebGPU

It is based on APIs provided by Vulkan, Metal, and Direct 3D 12.



Computer Graphics App Execution Process through WebGL:
The execution involves web servers and browsers.



Computer Graphics App Execution Process through WebGL:
The execution involves web servers and browsers.

Practical Pipeline Architecture of Computer Graphics



Vertices

1. Vertex Processing

2. Clipping and Primitive Assembly

3. Rasterization

4. Fragment Processing

Pixels



1. Vertex Processing

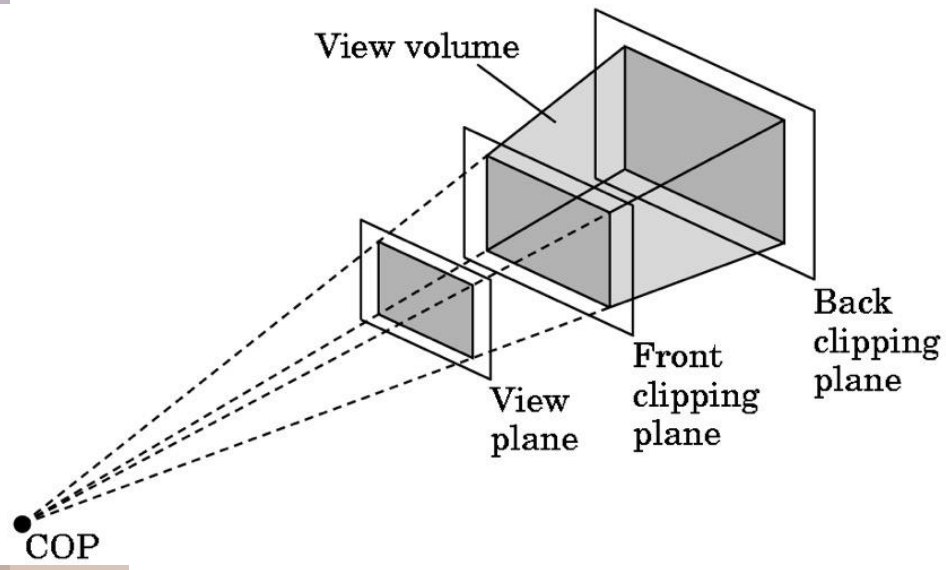


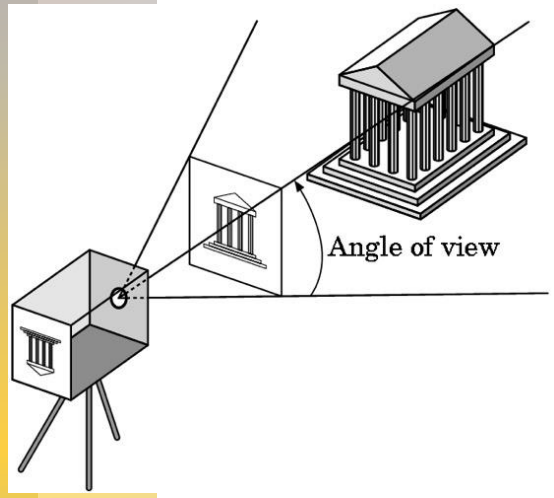
Most of the work executed in the Vertex Processing pipeline is **converting the object representation** from one coordinate system to another.

Any **change in the coordinates** is equivalent to a **matrix transformation**.

Projection (part of vertex processing)

- Projection is a process combining a 3-dimensional object **to produce a 2-dimensional image**.
- In Vertex Processing, the projection occurs **when the object coordinates are converted to camera/eye coordinates**.
- There are two types of projections:
- **Perspective projection**:
  All projectors meet at a central point of projection
- **Parallel projection**:
  Projectors are parallel, the center of projection is replaced by the direction of projection





2. Clipping and Primitive Assembly

- Just like cameras in the real world cannot “see” the whole scene, virtual cameras can **see only part of the scene** from the object space.
- Objects that are not in the view volume are considered to be clipped out of the view.

2. Clipping and Primitive Assembly

- Vertices need to be collected into geometric objects before the rasterization process.
- Basic geometric objects or what so-called primitive assembly include:

1. Point
2. Line
3. Polygon
4. Curve
5. Surface

3. Rasterization

- If an object is not clipped out, the corresponding pixels in the frame buffer **must already be colored**.
- The rasterizer produces a set of **fragments** for each object.
- Fragments are potential pixels, which:
  - Has a **location** in the frame buffer
  - Has **color** and **depth** attributes
- The vertex attributes are interpolated across the object by the rasterizer.

4. Fragment Processing

- The fragments that have been collected by the rasterizer are processed to **determine the color of pixels** associated with them in the frame buffer.
- The colors can be determined by:
- **Texture Mapping** or by
- **Interpolation** of vertex colors.
- A fragment may be covered by another fragment that is closer to the camera in depth. In this case, a **hidden surface removal** process needs to be carried out.

Introduction to WebGL

- WebGL = Graphics API for the web
- Three.js uses WebGL
- WebGL is more difficult to use than Three.js
- WebGL gives programmers full control over the hardware
- Two versions of WebGL:
  - WebGL 1.0 that is based on OpenGL ES 2.0
  - WebGL 2.0 that is based on OpenGL ES 3.0
- Two sides of WebGL programs:
  - Part written in JavaScript
  - Part written in GLSL that run on the GPU

The Programmable Pipeline

- Old OpenGL API uses a fixed-function pipeline
- Modern OpenGL introduced a programmable pipeline
- WebGL uses a programmable pipeline, and it is mandatory



The Programmable Pipeline



- The programs that are written as part of the pipeline are called shaders
  - Vertex shader: called once for each vertex in a primitive
  - Fragment shader: called once for each pixel in the primitive
- WebGL pipeline also contains several stages from the original fixed-function pipeline
  - For example, the **depth test** is still part of the fixed functionality, and it can be enabled or disabled in WebGL

The WebGL Graphics Context

- It is a JavaScript object whose methods implement the JavaScript side of the WebGL API
- A graphics context for WebGL 2.0 can be obtained by calling the function canvas.getContext("webgl2"), where canvas is a DOM object representing the canvas

```
canvas = document.getElementById("webglcanvas");
gl = canvas.getContext("webgl2");
if ( ! gl ) {
    throw new Error("WebGL not supported; can't create graphics context.");
}
…
<canvas width="800" height="600" id="webglcanvas"></canvas>
```

Sample `init` function called when the page is loaded (`window.onload = init`)

```
function init() {
    try {
        canvas = document.getElementById("webglcanvas");
        gl = canvas.getContext("webgl2");
        if ( ! gl ) {
            throw new Error("WebGL not supported.");
        }
    }
    catch (e) {
          // report the error
        return;
    }
    initGL();  // a function that initializes the WebGL graphics context,
               // including creating and installing the shader programs
      …  // other JavaScript initialization
}
```

The Shader Program

- Drawing with WebGL requires a shader program: a vertex shader and a fragment shader
- Shaders are written in GLSL programming language that is based on the C programming language
- The vertex shader and fragment shader are separate programs, each with its own main() function
- The two shaders are compiled separately and then "linked" to produce a complete shader program
- The JavaScript API for WebGL includes functions for compiling the shaders and then linking them

WebGL's Program Object

```
var program = gl.createProgram();

gl.attachShader( program, vertexShader
);

gl.attachShader( program,
fragmentShader );

gl.linkProgram( program );
```

- gl.useProgram(program);

- It is a **container** for shaders.
- It can load **many** shaders.
- It can load other GLSL functions beside of the main shader functions.

Passing Shaders from CPU to GPU

- Shaders are added to a program object to be compiled.
- The common way to pass any shaders from CPU to GPU is by firstly writing their source code as a string inside an HTML document or JS file. Afterwards, the string is passed through gl.shaderSource (shaderPointer, sourceCodeString) function.
- If the source code is in an HTML document, then we can obtain and input it into the JS application through **getElementById function**.
- The source code can also be written in a separate file. In this case, we may need to use a reader library such as Signals.js to convert the file into a string.

The Shader Program

- To use the functions, the source code for the shaders must be JavaScript strings
- Three steps to create the vertex shader

```
let vertexShader = gl.createShader( gl.VERTEX_SHADER );
gl.shaderSource( vertexShader, vertexShaderSource );
gl.compileShader( vertexShader );
```

- To check for compilation errors:

```
gl.getShaderParameter( vertexShader, gl.COMPILE_STATUS )
```

The Shader Program

- The fragment shader can be created in a similar way

```
let fragmentShader = gl.createShader( gl.FRAGMENT_SHADER );
gl.shaderSource( fragmentShader, fragmentShaderSource );
gl.compileShader( fragmentShader );
```

- The shaders need to be "attached" to the program object before linking:

```
let prog = gl.createProgram();
gl.attachShader( prog, vertexShader );
gl.attachShader( prog, fragmentShader );
gl.linkProgram( prog )
```

Typical Shader Creation Function

```
function createProgram(gl, vertexShaderSource, fragmentShaderSource) {
   let vsh = gl.createShader( gl.VERTEX_SHADER );
   gl.shaderSource( vsh, vertexShaderSource );
   gl.compileShader( vsh );
   if ( ! gl.getShaderParameter(vsh, gl.COMPILE_STATUS) ) {
      throw new Error("Error in vertex shader:  " + gl.getShaderInfoLog(vsh));
   }
   let fsh = gl.createShader( gl.FRAGMENT_SHADER );
   gl.shaderSource( fsh, fragmentShaderSource );
   gl.compileShader( fsh );
   if ( ! gl.getShaderParameter(fsh, gl.COMPILE_STATUS) ) {
      throw new Error("Error in fragment shader:  " + gl.getShaderInfoLog(fsh));
   }
   let prog = gl.createProgram();
   gl.attachShader( prog, vsh );
   gl.attachShader( prog, fsh );
   gl.linkProgram( prog );
   if ( ! gl.getProgramParameter( prog, gl.LINK_STATUS) ) {
      throw new Error("Link error in program:  " + gl.getProgramInfoLog(prog));
   }
   return prog;
}
```

Shader Programs

- Next step: tell the WebGL context to use the program
  - If `prog` is a program identifier returned by the previous function, call

```
gl.useProgram( prog );
```

- We can create several shader programs then switch from one program to another at any time by calling `gl.useProgram`, even in the middle of rendering an image
  - Three.js, for example, uses a different program for each type of Material
- Shaders and programs that are no longer needed can be deleted to free up the resources they consume

```
gl.deleteShader(shader)
gl.deleteProgram(program)
```

Adding a (new) Vertex Shader

- var vertexShader;
- var vertexElement = document.getElementById(vertexShaderId); // The source code is in the .html

- vertexShader = gl.createShader(gl.VERTEX_SHADER);
- gl.shaderSource(vertexShader, vertexElement.text);
- gl.compileShader(vertexShader);

- …

- // After the shader program has been created
- gl.attachShader(shaderProgram, vertexShader);

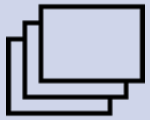
Shaders

- There are two types of shader: **Vertex** and **Fragment** shaders.
- Vertex shader should has at least a defined **gl_Position** variable.
- Fragment shader should has at least a defined **gl_FragColor** variable.
- Both of above shaders are full-state and independent programs.
- Number **2**, **3**, **4**, on **vec**tor or **mat**rix data type represents dimension.
- A **precision** config should be declared globally in the Fragment shader.

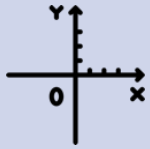
Shaders and Coordinate System



Vertex shader results in output in Clip Coordinates (3D).



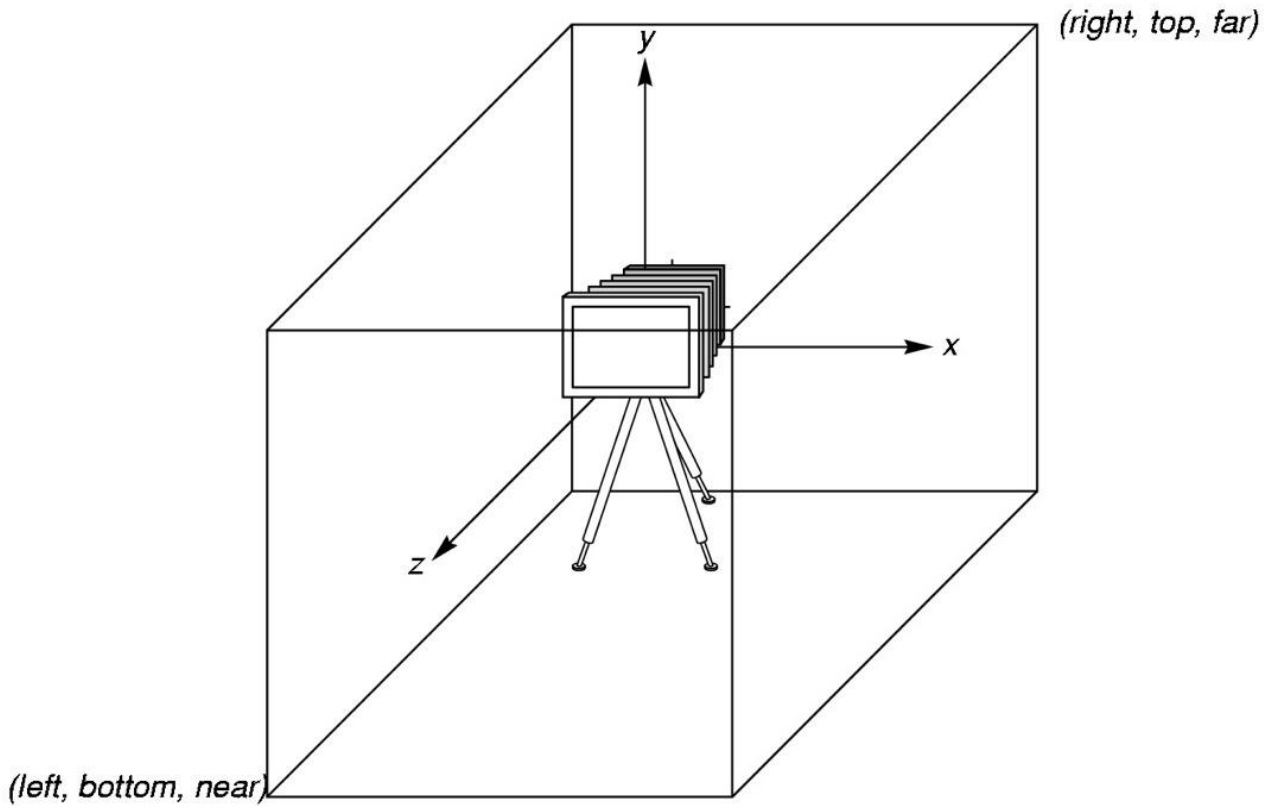
Input to Fragment shader from Rasterizer is in the form of Window Coordinates (2D).



JS application may provide vertices data in any kinds of coordinate system. However, in the end, vertex shaders should produce gl_Position in the context of Clip Coordinates (3D).

Shader's Camera

- Shaders locate a **camera** in the **origin (0,0)** of object space.
- The camera face towards the **negative of z axis**.
- The camera's **viewing volume** is a **2x2x2 cube** with the origin as its center.



GL Shading Language

01

It is like C language, but equipped with special matrix and vector data types

02

Similar to Nvidia's Cg and Microsoft HLSL

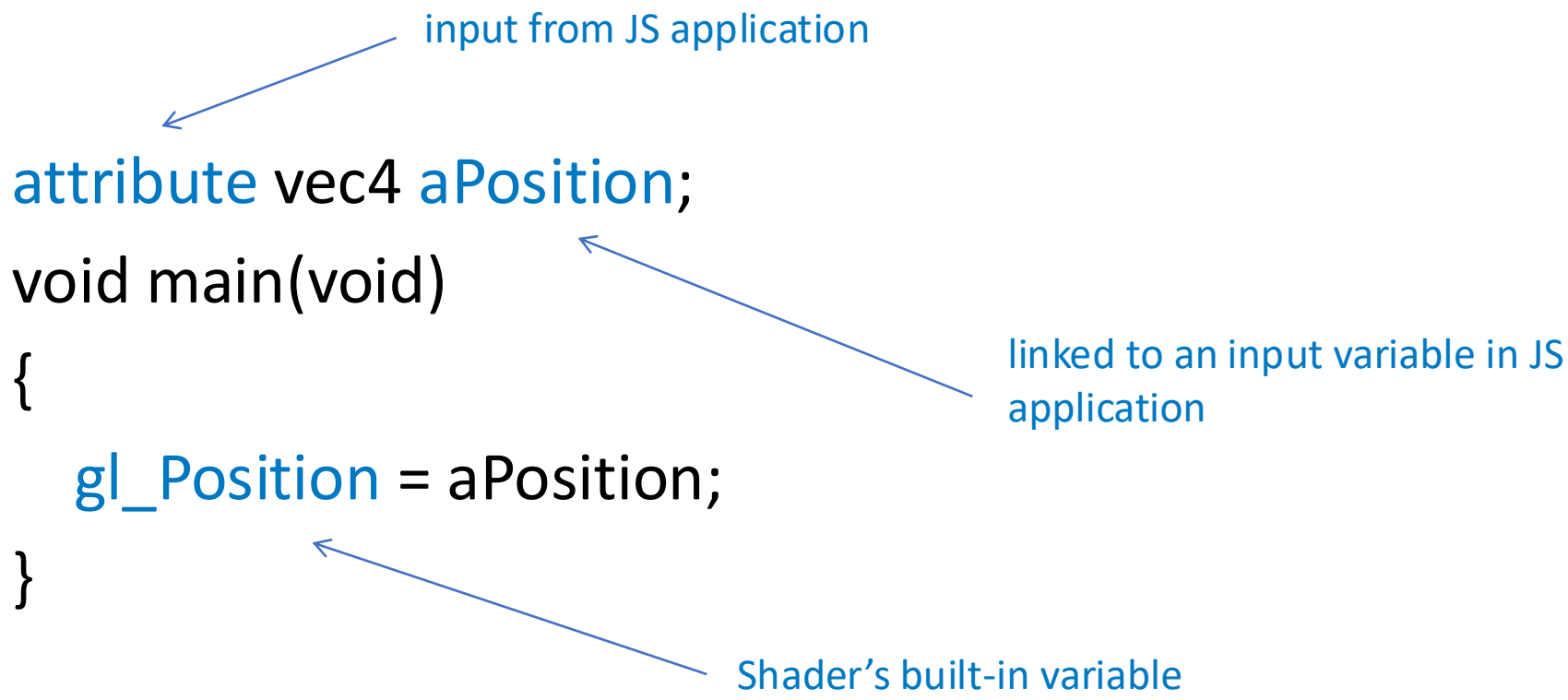
03

The source codes are sent from JS application to the GLSL shaders by the help of WebGL functions.

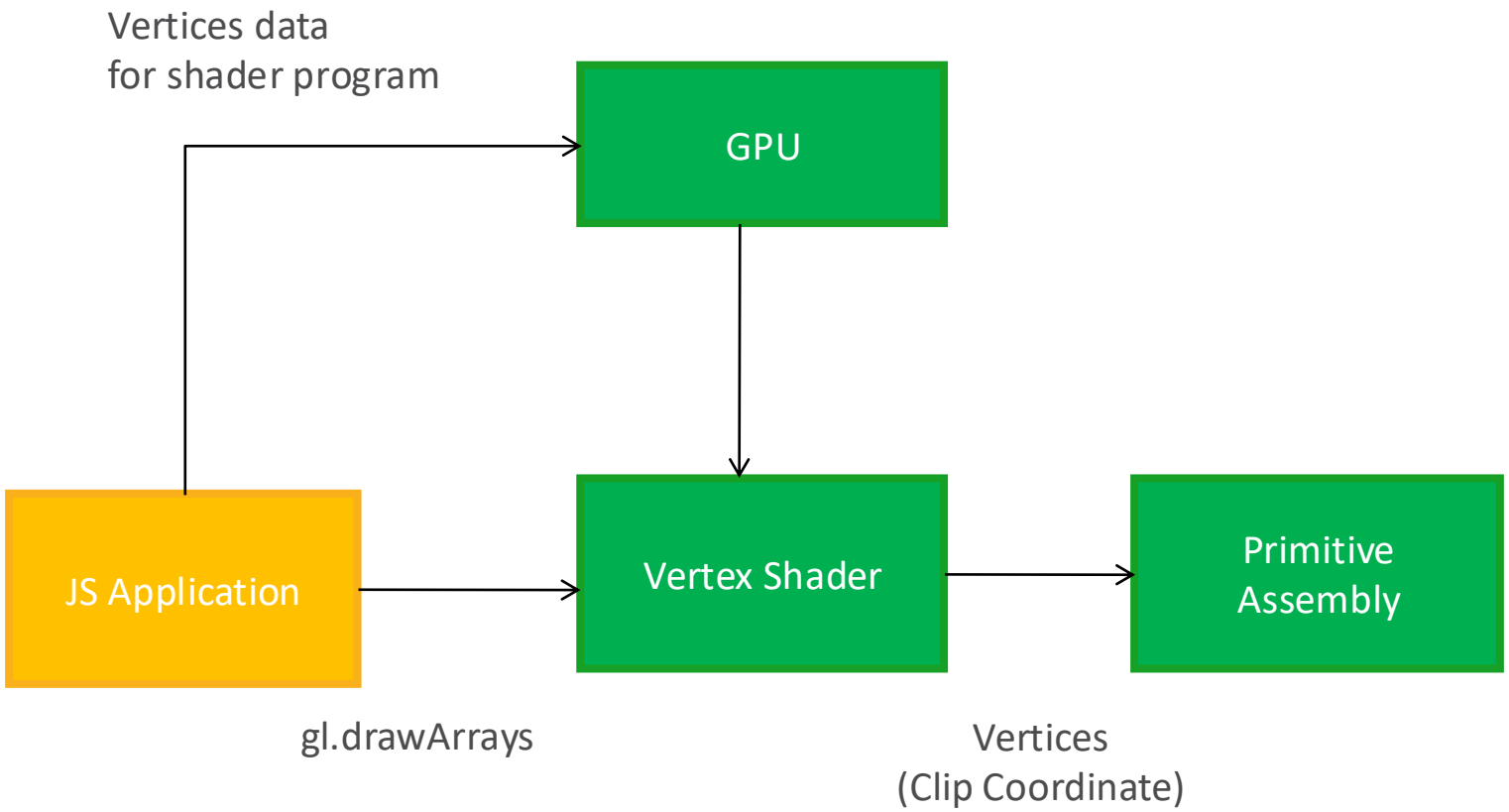
04

WebGL functions compile, link and provide information to the GLSL shaders.

Simple Vertex Shader



Vertex Shader Execution Model



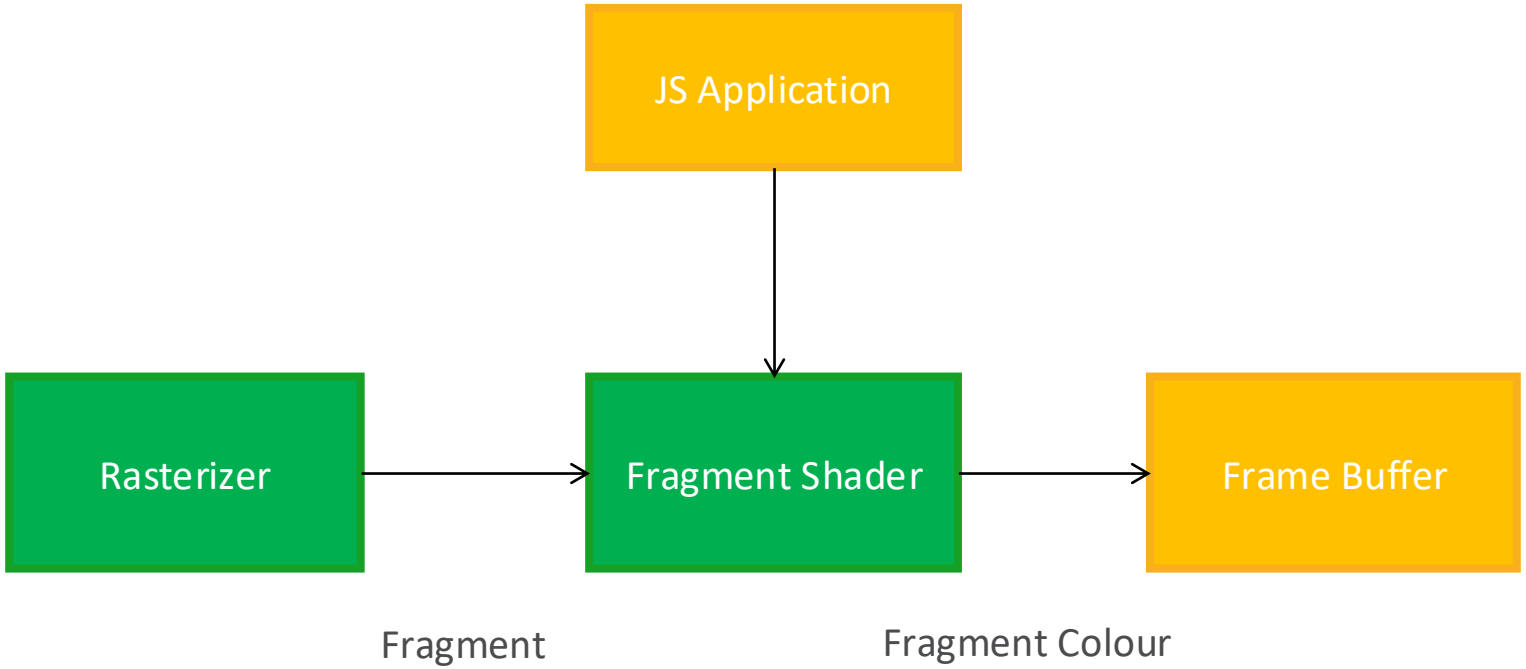
Simple Fragment Shader

precision configuration:
how much detail the colour interpolation

```
precision mediump float;
void main(void)
{
    gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0);
}
```

Shader's built-in variable

Fragment Shader Execution Model



GLSL Data Types

- Just like C language: **float**, **int**, **bool**
- Vector specific:
  float: **vec2**, **vec3**, **vec4**
  int: **ivec2**, **ivec3**, **ivec4**
  bool: **bvec2**, **bvec3**, **bvec4**
- Matrix specific: **mat2**, **mat3**, **mat4**
  with the standard reference of: *m[column][row]*
- Constructor function behaves like C language:
  vec3 a = vec3 (1.0, 2.0, 3.0);

GLSL Qualifiers

- A primitive is defined by its **type** and by a list of **vertices**
- There are two general categories of data that can be provided for the primitive
  - **Uniform** variables: have a single value that is the same for the entire primitive
  - **Attribute** variables: have the value that can be different for different vertices
- Example
  - Vertex coordinates (uniforms or attributes?)
  - Vertex colors (can be both?)
  - Normal vectors (?)
  - Material properties (?)
  - Texture coordinates (?)
  - Geometric transform (?)

GLSL Qualifiers

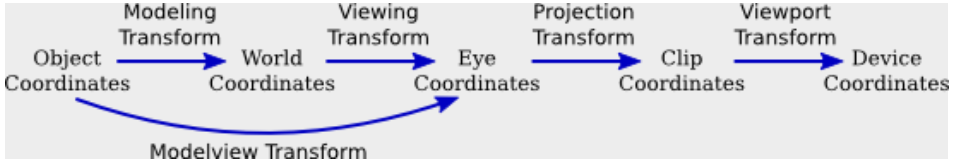
- WebGL does not come with **any** predefined attributes
- In WebGL, attributes are values that are passed into the vertex shader
- Uniforms can be passed into the vertex shader, the fragment shader, or both

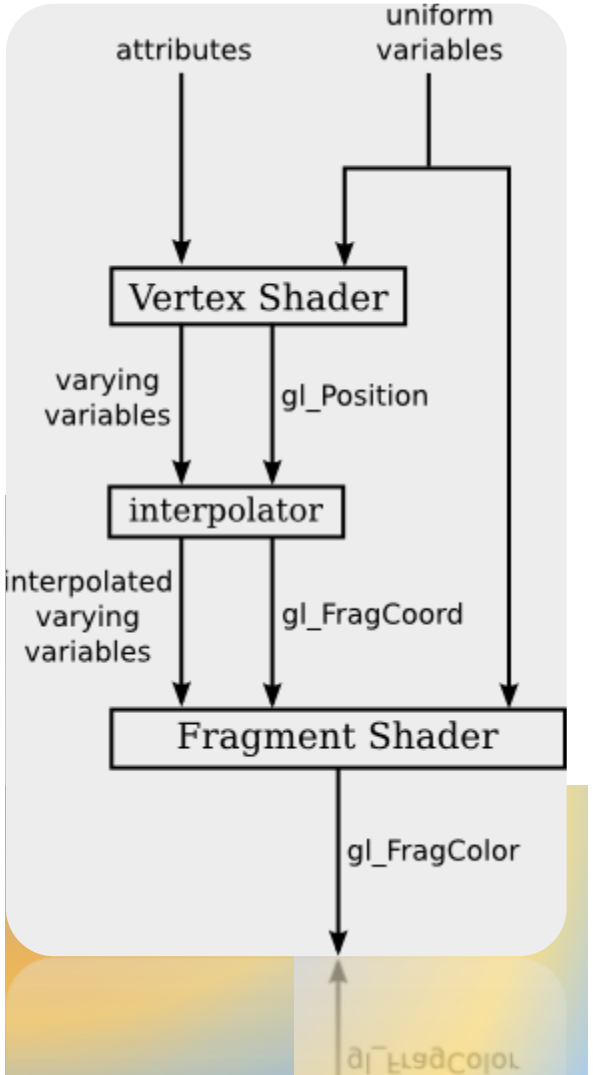
Drawing an object: JavaScript side

- When drawing a primitive, the JavaScript program specifies values for any attributes and uniforms in the shader program.
- It must send these values to the to the GPU before drawing the primitive.
- The primitive can then be drawn by calling a single JavaScript function.
- At that point, the GPU takes over, and executes the shader programs.

Drawing an object: GPU side

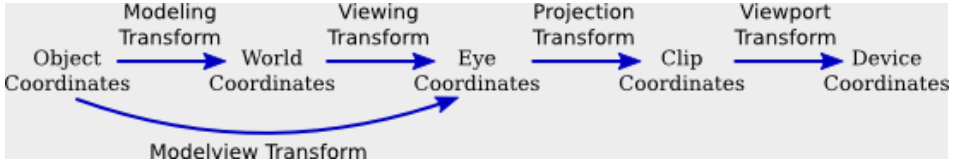
- When drawing the primitive, the GPU calls the vertex shader once for each vertex.
- The attribute values for the vertex that is to be processed are passed as input into the vertex shader.
- Values of uniform variables are also passed to the vertex shader.
- The way this works is that both attributes and uniforms are represented as global variables in the vertex shader program.
- Before calling the shader for a given vertex, the GPU sets the values of those variables appropriately for that specific vertex.

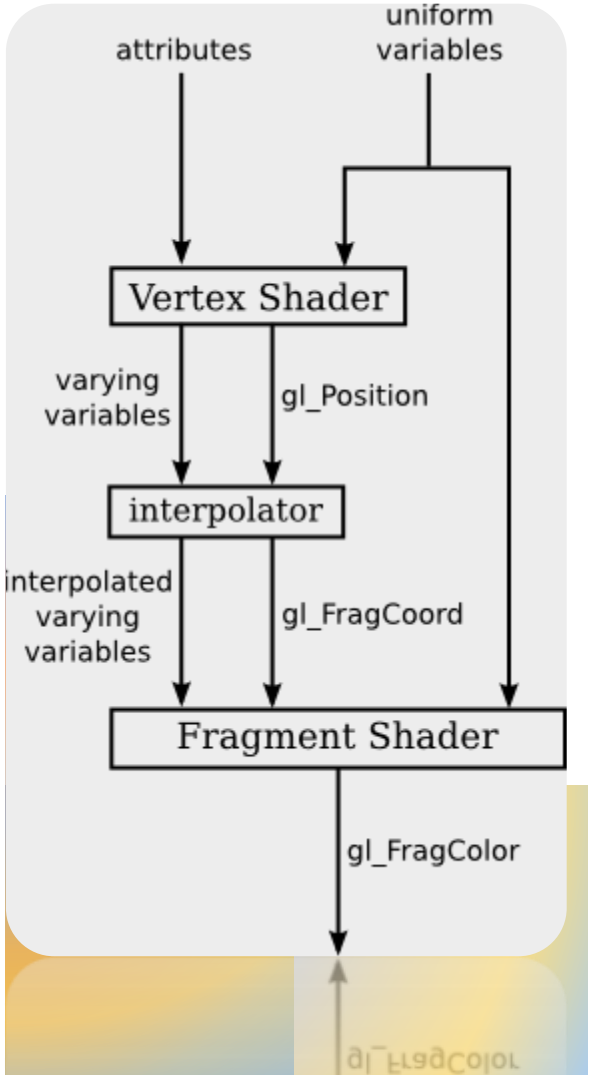




Vertex shader program

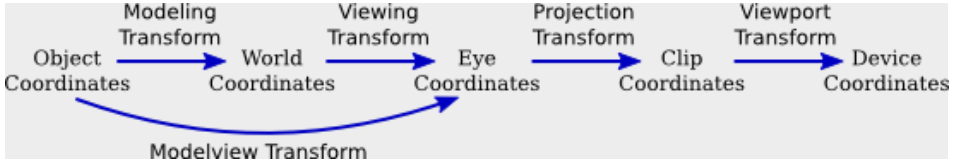
- As one of its outputs, the vertex shader must specify the coordinates of the vertex in the clip coordinate system by assigning a value to a special variable named **`gl_Position`**
- The position is often computed by applying a transformation to the attribute that represents the coordinates in the object coordinate system, but exactly how the position is computed is up to the programmer

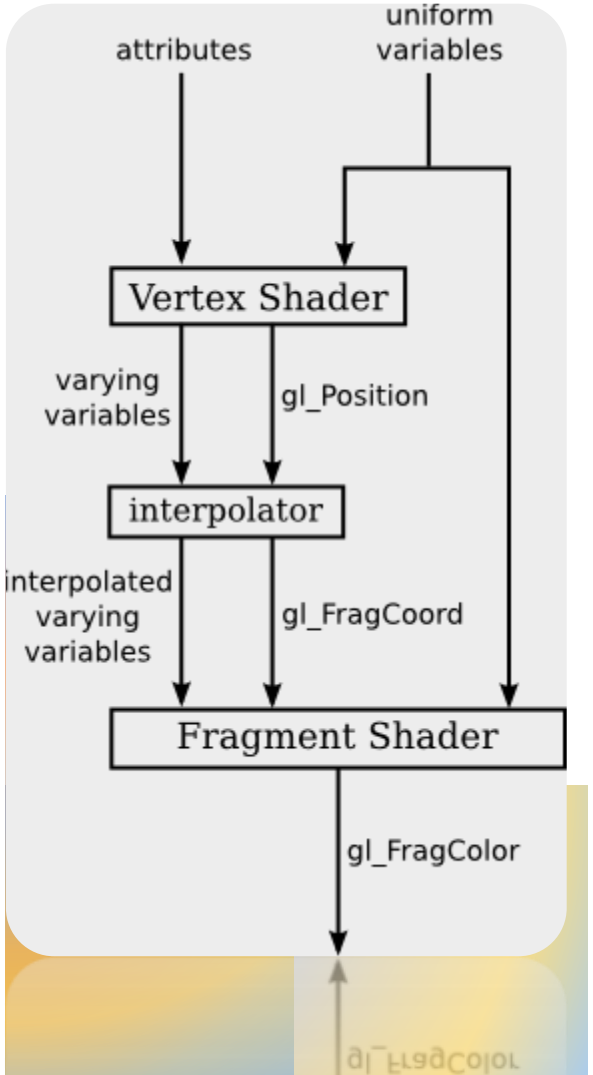




Vertex shader program

- After the positions of all the vertices in the primitive have been computed, a fixed-function stage in the pipeline **clips** away the parts of the primitive whose coordinates are outside the range of valid clip coordinates (−1 to 1 along each coordinate axis).
- The primitive is then rasterized; that is, it is determined which pixels lie inside the primitive.
- The GPU then calls the fragment shader once for each pixel that lies in the primitive.

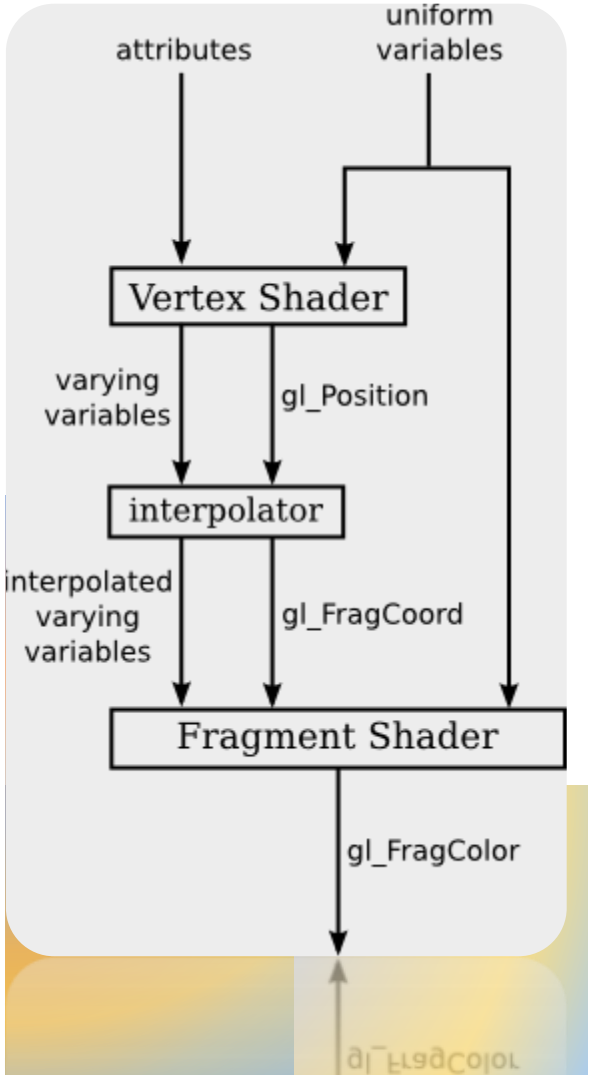




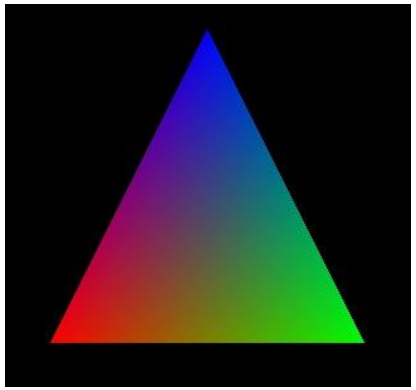
Fragment shader program

- The fragment shader has access to uniform variables (but not attributes).
- It can also use a special variable named `gl_FragCoord` that contains the clip coordinates of the pixel.
- Pixel coordinates are computed by interpolating the values of gl_Position that were specified by the vertex shader.
- The interpolation is done by another fixed-function stage that comes between the vertex shader and the fragment shader.

Varying variables

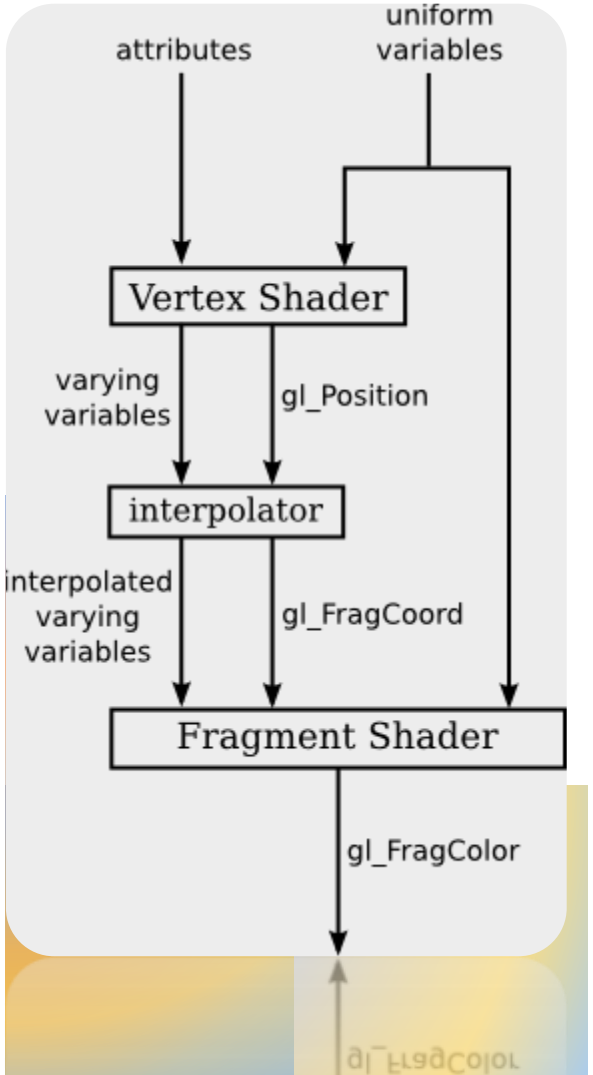


- In OpenGL, the color of an interior pixel is computed by interpolating the color at the vertices.



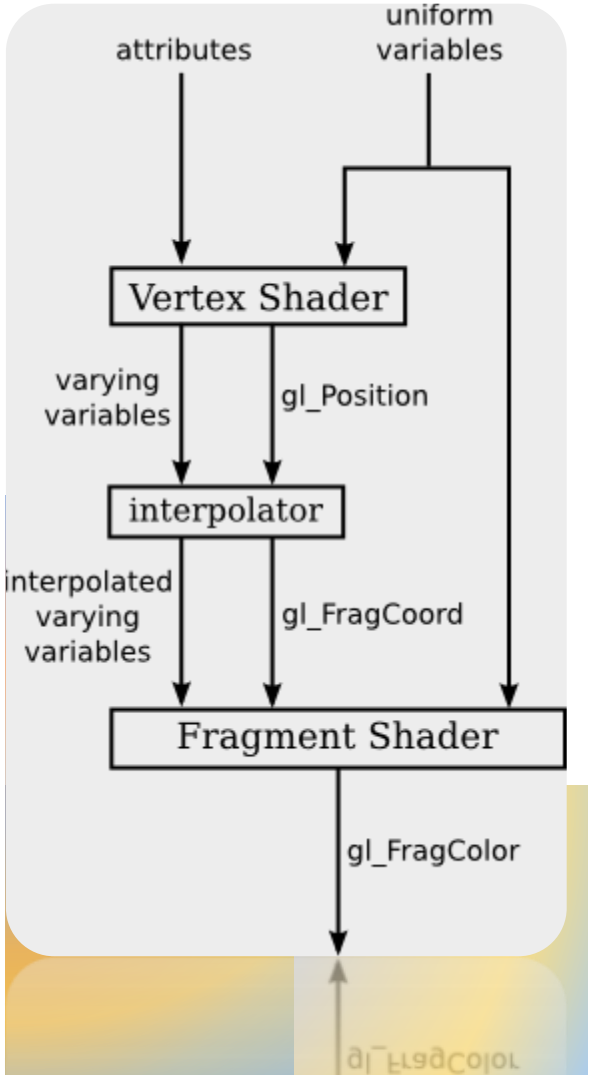
- In GLSL, this is implemented using varying variables.

Varying variables



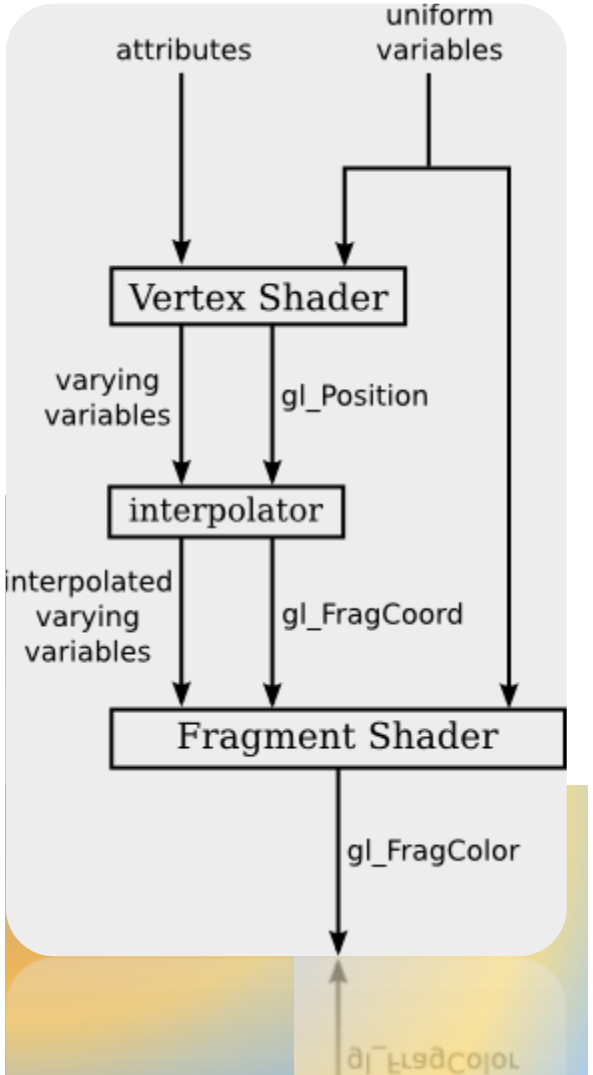
- A varying variable is declared both in the vertex shader and in the fragment shader.
- The vertex shader assigns a value to the varying variable.
- The interpolator interpolates the values to produce a value for each pixel.
- When the fragment shader is executed for a given pixel, the value of the varying variable is the interpolated value for that pixel.

Varying variables



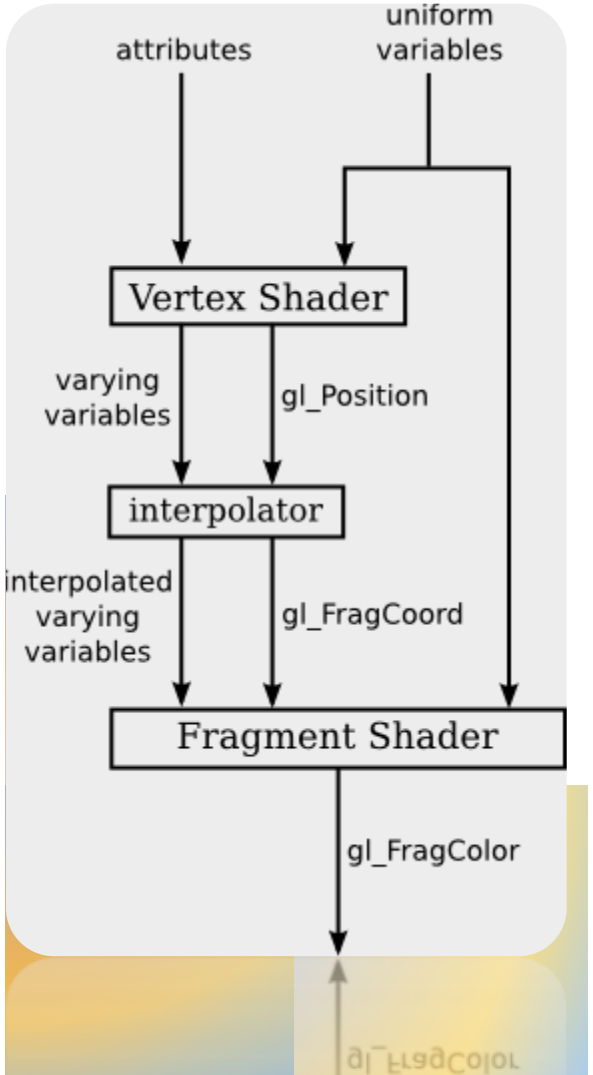
- Varying variables communicate data from the vertex shader to the fragment shader.
- They are defined in the shader source code, not used or referred to in the JavaScript side of the API.
- Note that it is entirely up to the programmer to decide what varying variables to define and what to do with them.

Fragment Shader



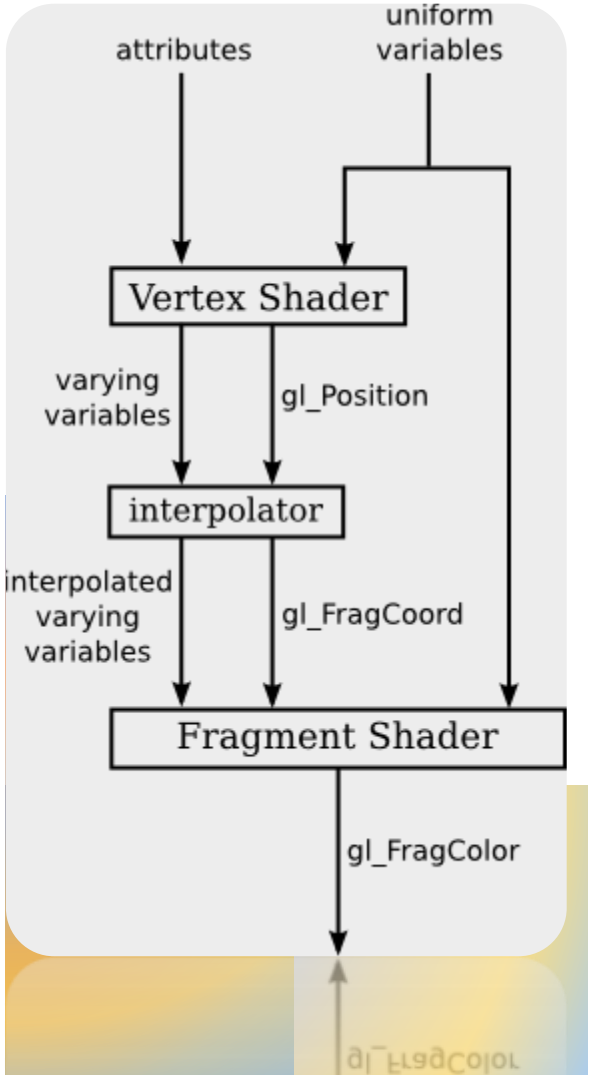
- The job of the fragment shader is simply to specify a color for the pixel.
- It does that by assigning a value to a special variable named **`gl_FragColor`**.
- That value will then be used in the remaining fixed-function stages of the pipeline.

Summarize



- The JavaScript side sends values for attributes and uniform variables to the GPU and then issues a command to draw a primitive.
- The GPU executes the vertex shader once for each vertex.
- The vertex shader can use the values of attributes and uniforms.
  - It assigns values to gl_Position and to any varying variables that exist in the shader.
- After clipping, rasterization, and interpolation, the GPU executes the fragment shader once for each pixel in the primitive.
- The fragment shader can use the values of varying variables, uniform variables, and gl_FragCoord.
  - It computes a value for gl_FragColor.

Notes on GLSL ES 3.00



- There is no special variable gl_FragColor.
  - The fragment shader must define its own output variable to represent the color.
- Attribute variables become "in" variables, since they are inputs to the vertex shader, and varying variables become "out" variables in the vertex shader and "in" variables in the fragment shader. The variable gl_FragColor is replaced by an "out" variable in the fragment shader

GLSL Qualifiers

Attribute

Become the inputs for Vertex shaders

Declared once globally in Vertex shader, and cannot be modified respectively

Lead the shader's variable's value varies across vertices

Varying

As the outputs of Vertex shaders; and as the inputs for Fragment shaders

Declared twice globally: One is in Vertex shader, the other one is in Fragment shader

Interpolated automatically by Rasterizer

Uniform

Become the inputs for any or both shaders

Lead the shaders' variable's value remain the same across vertices and fragments

Cannot be modified just like other qualifiers. To change its value across the frames, we need to modify its corresponding variable in JS application

Shaders' Source Code

Vertex Shader

```
attribute vec4 aPosition;
attribute vec4 aColor;
varying vec4 vColor;

void main()
{
        gl_Position = aPosition;
        vColor = aColor;
}
```

Fragment Shader

```
precision mediump float;
varying vec4 vColor;
uniform vec4 uAmbient;

void main()
{
    gl_FragColor = uAmbient + vColor;
}
```

Let's have some naming conventions for both GLSL and JS applications:

1. All the names of **attribute** variables are started with **a**.
2. All the names of **varying** variables are started with **v**.
3. All the names of **uniform** variables are started with **u**.

Sending Colors as an Attribute

```
var vertexBuffer = gl.createBuffer();

gl.bindBuffer( gl.ARRAY_BUFFER, vertexBuffer );

gl.bufferData( gl.ARRAY_BUFFER, new Float32Array (colors), gl.STATIC_DRAW );


var aColor = gl.getAttribLocation( program, ”aColor" );

gl.vertexAttribPointer( aColor, 3, gl.FLOAT, false, 0, 0 );

gl.enableVertexAttribArray( aColor );
```

Sending Colors as a Uniform

in JS Application

```
vec4 color = vec4(1.0, 0.0, 0.0, 1.0);

uColor = gl.getUniformLocation( program, "uColor" );

gl.uniform4f( uColor, color);
```

In GLSL Shader

```
uniform vec4 uColor;

void main()

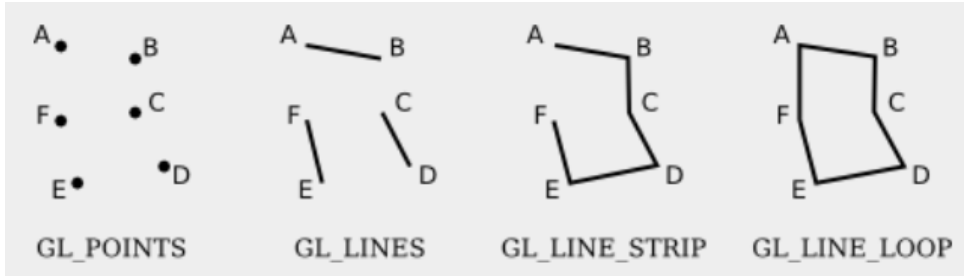
{

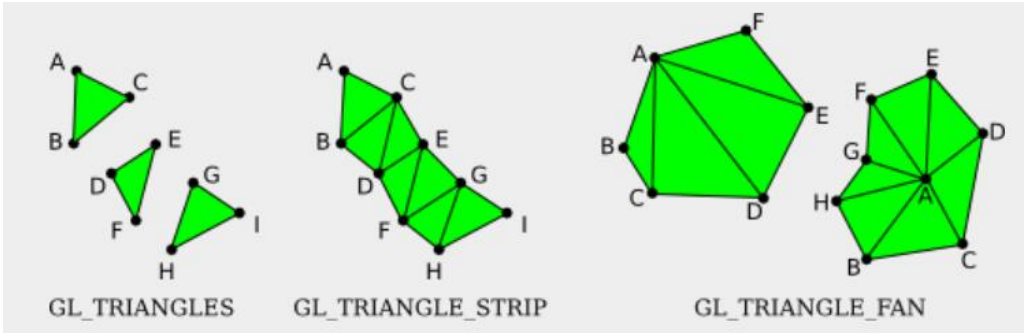
    gl_FragColor = uColor;

}
```

Primitive objects in WebGL

- In WegGL, the seven types of primitive are identified by the constants
  - `gl.POINTS`
  - `gl.LINES`
  - `gl.LINE_STRIP`
  - `gl.LINE_LOOP`
  - `gl.TRIANGLES`
  - `gl.TRIANGLE_STRIP`
  - `gl.TRIANGLE_FAN`





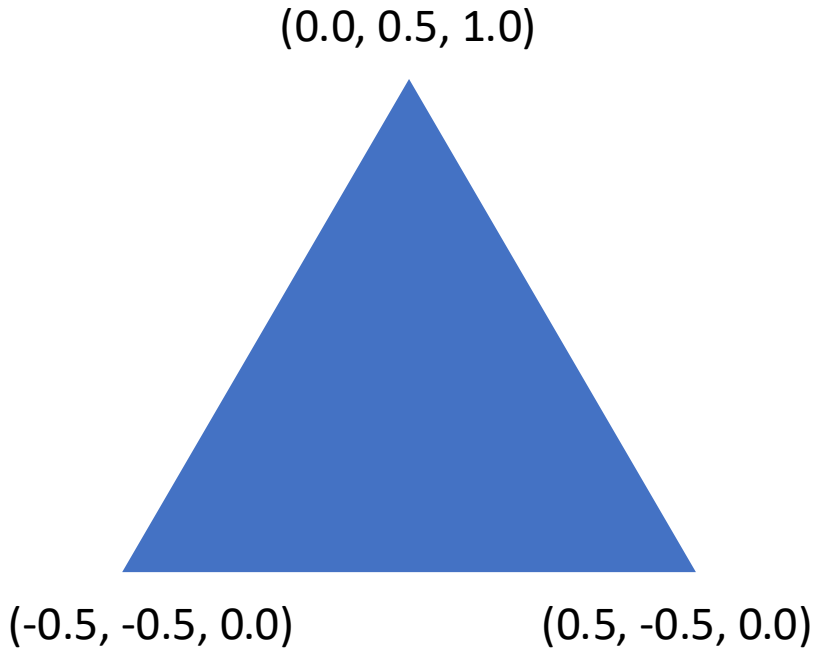
where `gl` is a WebGL graphics context.

How to Draw a Triangle

1. Put the triangle geometry data into an array

```
var points = [
        vec3 (-0.5, -0.5, 0.0),
        vec3 (  0.5, -0.5, 0.0),
        vec3 (  0.0,   0.5, 1.0)
];
```

2. Send the array to GPU
3. Tell GPU to draw the triangle



Next Week Topic

Example 1 and More on WebGL & GLSL

